# CricketAI: An Intelligent Virtual Cricket Coach Using Pose Estimation and Retrieval-Augmented Generation


Abdul Wahid H
*Dept. of Computer Science and Engineering*
*VISTAS*
Chennai, India
Reg. No. 24623102
24623102@vistas.ac.in

Sarveshwaran K
*Dept. of Computer Science and Engineering*
*VISTAS*
Chennai, India
Reg. No. 24623153
24623153@vistas.ac.in

Vignesh P
*Dept. of Computer Science and Engineering*
*VISTAS*
Chennai, India
Reg. No. 24623503
24623503@vistas.ac.in

Dr. N. Meenakshi
*Dept. of Computer Science and Engineering*
*VISTAS*
Chennai, India
meenakshin.se@vistas.ac



***Abstract*—Personalised technique feedback is difficult for most amateur cricketers to obtain: qualified coaches are scarce, recorded video needs expert interpretation, and general-purpose chatbots are not grounded in curated coaching knowledge. This paper presents CricketAI, a web-based virtual cricket coach that integrates single-camera pose analysis, a retrieval-augmented generation (RAG) assistant and structured skill lessons in one application. The video-analysis pipeline decodes an uploaded batting video with OpenCV, estimates 33 body landmarks per frame with MediaPipe Pose, derives geometric measurements such as joint angles, and converts them through rule-based evaluation into a technical score, a confidence value and a per-metric report. The coaching assistant embeds user questions with BAAI bge-base-en-v1.5, retrieves candidate passages from a ChromaDB store of approximately 1,002 cricket knowledge chunks, reorders them with a BGE cross-encoder reranker and generates grounded answers with a locally hosted Qwen3:4B model, so user queries are not sent to external AI services. The system is implemented with a React–TypeScript frontend and a FastAPI–MongoDB backend secured with JSON Web Tokens. Functional validation confirmed end-to-end execution of authentication, dashboard, video upload and retrieval, and coaching-query operations, and a complete analysis of a 1,654-frame clip recorded at 30.03 frames per second. We separate geometric measurements from coaching judgements, discuss the limits of monocular pose estimation, and define an evaluation protocol covering pose-measurement error, retrieval Recall@k, expert-rated answer quality, latency and usability as the basis for validation.**



***Index Terms*—cricket coaching, human pose estimation, MediaPipe, retrieval-augmented generation, reranking, large language models, sports analytics**


## I. Introduction

Batting technique in cricket depends on the coordinated position of the head, shoulders, elbows, hips, knees and feet through the stance, backlift, downswing and follow-through. Players improve this coordination mainly through repeated observation and correction by a coach. For most school, college and club players in India and elsewhere, such feedback is occasional at best: qualified coaches supervise many players at once, academies are costly or distant, and verbal feedback is rarely recorded in a form that can be compared over time.

Smartphones have made video recording of practice routine, but interpreting a recording still requires expertise, and a single camera captures only a two-dimensional projection of three-dimensional movement. Human pose estimation offers a way to extract consistent, quantitative descriptions of body position from ordinary video [1], [2], and MediaPipe Pose makes this feasible in real time on commodity hardware [3], [4]. In parallel, retrieval-augmented generation (RAG) has emerged as a practical way to ground language-model answers in a domain corpus [5], [6], reducing, though not eliminating, the risk of confident but incorrect answers [7].

Prior cricket vision research has largely addressed stroke *classification* [8], [9], answering which shot was played rather than how it was executed, and has not been combined with a conversational assistant that can explain technique. This paper presents **CricketAI**, a web application that brings together pose-based video analysis, a locally hosted RAG coaching assistant and structured skill lessons behind a single authenticated interface. The contributions are:

1) An integrated system architecture combining a MediaPipe/OpenCV batting-analysis pipeline, a retrieve–rerank–generate assistant and a lesson module, with per-user storage of results.
2) A video-analysis workflow that converts single-camera batting video into recorded pose-derived measurements, a technical score, a confidence value and a per-metric report.
3) A fully local RAG pipeline using bge-base-en-v1.5 embeddings, ChromaDB, a BGE cross-encoder reranker and Qwen3:4B, avoiding external AI services.
4) An explicit separation of geometric measurements from coaching judgements, and an evaluation protocol for measurement error, retrieval quality, answer quality, latency and usability.

Section II reviews related work. Section III presents the methodology, Section IV the system architecture and Section V the application workflow. Section VI reports results and discussion, and Section VII concludes.

## II. Related Work

### A. Computer Vision and Pose Estimation in Sport

Thomas *et al.* [10] surveyed computer vision in sport and noted that many deployed systems rely on controlled multi-camera installations, whereas technique analysis from ordinary footage remains difficult because of occlusion, motion blur and varying viewpoints. Deep-learning pose estimation began with direct regression of joint coordinates in DeepPose [11]; OpenPose [12] introduced part affinity fields for real-time multi-person 2D estimation; and Zheng *et al.* [2] survey the

subsequent 2D and 3D methods, datasets and metrics. BlazePose [4] uses a lightweight detector–tracker design that predicts 33 keypoints for a single person in real time on a mobile phone, and underlies the MediaPipe Pose solution [3], [13]. A systematic review of camera-based pose estimation in sport and exercise [1] highlighted recurring concerns about validation against reference measurement systems.

### *B. Cricket Stroke and Technique Analysis*

Sen *et al.* [8] proposed CricShotClassify, a CNN–GRU model that classifies batting shots from video clips. Siddiqui *et al.* [9], the base work for this project, extracted MediaPipe pose features from videos of eight strokes (pull, cut, cover drive, straight drive, backfoot punch, on drive, flick and sweep) and compared several classifiers, reporting a best accuracy of 99.77% with a random forest and a k-fold accuracy of 95.0%. These studies show that pose features carry discriminative information about batting, but their output is a stroke label rather than learner-facing technique feedback.

### *C. Retrieval-Augmented Generation and Reranking*

Lewis *et al.* [5] combined a parametric generator with a dense document index accessed by a neural retriever, improving open-domain question answering and the factuality of generated text. Dense passage retrieval [14] and Sentence-BERT [15] established embedding-based semantic search, and HNSW graphs [16] make approximate nearest-neighbour search over such embeddings efficient. Nogueira and Cho [17] showed that a BERT cross-encoder substantially improves passage ranking when used to rerank a candidate list. The BGE embedding family [18] and the Qwen3 model family [19] provide open models suitable for local deployment. Gao *et al.* [6] survey RAG pipelines that add reranking and context construction around retrieve-then-generate.

### *D. Intelligent Tutoring and AI Coaching*

VanLehn [20] found step-based intelligent tutoring systems nearly as effective as human tutoring in the reviewed studies, largely in mathematics and science. Although these findings cannot be assumed to transfer to motor skills, they support providing specific, learner-centred feedback tied to the task being practised.

### *E. Research Gap*

Table I compares the most relevant works. Existing cricket studies classify strokes but do not turn pose data into interpretable learner reports; pose-based sports tools are rarely combined with a domain-grounded assistant; and few accessible tools state clearly which outputs are measurements and which are judgements. CricketAI addresses these gaps in a single locally deployed application.

TABLE I
COMPARISON OF RELATED WORK

| Ref. | Method | Output | Gap w.r.t. CricketAI |
|---|---|---|---|
| [9] | MediaPipe features; RF, SVM, KNN, DT, LR, LSTM | Stroke label (8 classes) | No technique report or assistant |
| [8] | CNN + GRU on frames | Stroke label | No pose-level explanation |
| [4] | BlazePose detector–tracker | 33 keypoints | Generic; no cricket logic |
| [12] | Part affinity fields | Multi-person 2D pose | Heavier; no coaching layer |
| [1] | Systematic review | Survey | Stresses validation needs |
| [5] | Retriever + generator | Grounded text | Not sports-specific |
| [17] | BERT cross-encoder | Reranked passages | Component only |
| CricketAI | MediaPipe + rule-based metrics; BGE + ChromaDB + reranker + Qwen3:4B | Score, report, grounded answers, lessons | Validation pending (Sec. VI) |

## III. METHODOLOGY

CricketAI combines three methods: a pose-based video-analysis pipeline that turns a batting video into measurements and a report, a retrieve–rerank–generate assistant that answers technique questions from a curated knowledge base, and a structured lesson module. This section describes each method; Section IV shows how they are deployed and Section V how a user moves through them.

### *A. Video-Analysis Pipeline*

The pipeline is shown in Fig. 1. The uploaded file is validated and stored, then opened with OpenCV, which also reports the frame count and source frame rate. Each decoded frame is converted from OpenCV's BGR order to RGB and passed to MediaPipe Pose, which returns 33 landmarks with normalised coordinates $x, y \in [0, 1]$, a relative depth $z$ and a visibility score [13]. Because $x$ and $y$ are normalised separately by frame width $W$ and height $H$, landmarks are first converted to pixel coordinates so that angles are not distorted in non-square frames:

$$p_k = ( x_k \cdot W , y_k \cdot H ) \quad (1)$$

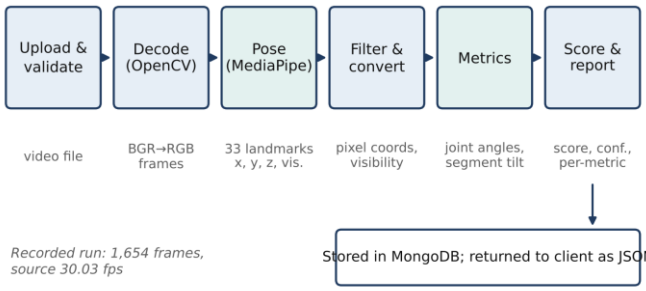



Fig. 1. Video-analysis pipeline from upload to stored report.

For three landmarks $A$, $B$, $C$ with vertex $B$ (Fig. 2), the joint angle is computed from the vectors $\mathbf{u} = A - B$ and $\mathbf{v} = C - B$:

$$\theta = \arccos( (u \cdot v) / (\|u\| \, \|v\|) ) \cdot 180/\pi \quad (2)$$

The cosine is clipped to $[-1, 1]$ before the inverse cosine, and frames in which either vector has near-zero length, or in which the required landmarks have low visibility, are excluded from that measurement. Segment orientation relative to the image vertical, for example of the trunk, is obtained analogously as $\varphi = \text{atan2}(|\Delta x|, |\Delta y|)$.

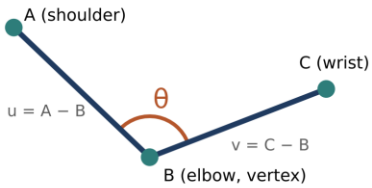



Fig. 2. Joint angle θ at landmark B formed by landmarks A and C.

These quantities are **geometric measurements** of the 2D image. Whether a measured angle is appropriate for a particular stroke, phase, player and camera viewpoint is a **coaching judgement** that requires reference data and expert validation; a projected angle also differs from the true 3D joint angle whenever the limb is not parallel to the image plane.

### *B. Technical Score and Report*

The analyzer compares each measurement with configured reference values and aggregates the per-metric results into a technical score and a confidence value. The report presents the video properties, score, confidence, per-metric values and results, and feedback text. In the current version the feedback is generated from predefined templates keyed to the metric results rather than by the language model. The score is therefore a rule-based indicator of agreement with the configured references; it has not yet been validated against assessments by qualified coaches and should not be read as a measure of batting ability.

### *C. RAG Coaching Assistant*

The assistant (Fig. 3) is built in two stages. Offline, cricket coaching material is cleaned, divided into chunks and embedded with bge-base-en-v1.5, which produces 768-dimensional vectors [18], [21]; the vectors, texts and metadata are stored in a persistent ChromaDB collection reported to hold approximately 1,002 chunks. Online, the question $q$ is embedded with the same model and the $k$ nearest chunks $d$ are retrieved by cosine similarity using ChromaDB's HNSW index [16]:

$$\mathrm{sim}(q, d) = (q \cdot d) \,/\, (\|q\|\,\|d\|) \qquad (3)$$

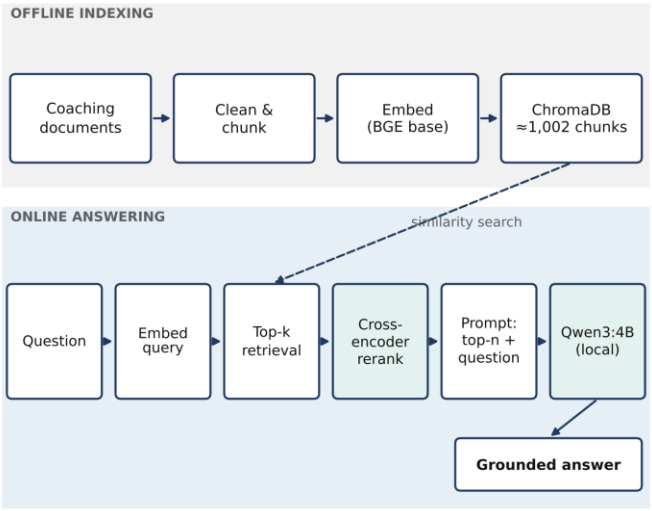



Fig. 3. RAG pipeline: offline indexing (top) and online retrieve–rerank–generate answering (bottom).

Because bi-encoder similarity can miss fine-grained relevance, the $k$ candidates are re-scored by a BGE cross-encoder $g$ that reads the question and passage jointly [17], and the top $n < k$ are kept:

$$r_i = g([q\,;\,d_i]),\quad i = 1, \ldots, k \qquad (4)$$

The selected passages and the question are placed in a prompt that instructs Qwen3:4B [19] to answer as a cricket coaching assistant using only the supplied context and to say when the context is insufficient. All inference runs locally.

### *D. Skill-Learning Module*

The skill-learning module lists cricket skills and opens each as a structured lesson with instructional content and practice guidance, giving learners an ordered path through technique topics alongside the analysis and question-answering features.

## IV. System Architecture

### *A. Layered Design*

CricketAI follows a four-layer client–server architecture (Fig. 4). The **presentation layer** is a React and TypeScript single-page application built with Vite and styled with Tailwind CSS. It calls the **application layer**, a FastAPI service run by Uvicorn [22], over HTTP with JSON payloads and a bearer JSON Web Token (JWT) [23]. Pydantic schemas validate every request and response. Four route groups handle authentication, dashboard and weekly progress, video upload and retrieval, and coaching queries.

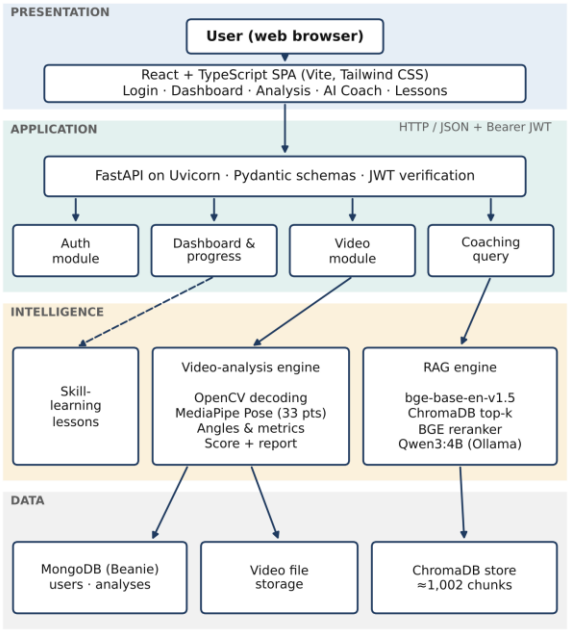



Fig. 4. Four-layer architecture of CricketAI. The video-analysis and RAG engines form the intelligence layer; MongoDB, video storage and ChromaDB form the data layer.

The **intelligence layer** contains two engines. The video-analysis engine uses OpenCV [24] and MediaPipe Pose to produce measurements, a score and a report. The RAG engine embeds questions with bge-base-en-v1.5 [21], retrieves candidates from ChromaDB [25], reranks them with a BGE

cross-encoder and generates answers with Qwen3:4B served locally through Ollama [26]. The **data layer** comprises MongoDB accessed through the Beanie object-document mapper [27] for users and analysis records, file storage for uploaded videos, and the persistent ChromaDB collection. Table II summarises the technology stack.

TABLE II
TECHNOLOGY STACK

| Layer | Components |
|---|---|
| Frontend | React, TypeScript, Vite, Tailwind CSS |
| Backend | Python, FastAPI, Uvicorn, Pydantic, JWT |
| Database | MongoDB with Beanie ODM |
| Vision | OpenCV, MediaPipe Pose (33 landmarks) |
| Retrieval | BAAI/bge-base-en-v1.5, ChromaDB, BGE reranker |
| Generation | Qwen3:4B via Ollama (local) |

### *B. Implementation Details*

The backend registers the Beanie document models at start-up and loads the embedding model, reranker and model-runtime connection once for reuse across requests. Development surfaced dependency conflicts among the transformer-based libraries used for embedding and reranking; these were resolved by pinning transformers 4.56.2, tokenizers 0.22.1, sentence-transformers 5.1.0 and FlagEmbedding 1.4.0, after which the coaching-query route executed successfully. FastAPI's automatically generated OpenAPI documentation was used for endpoint testing. On the frontend, a shared API helper attaches the stored JWT to each protected request and redirects to the login page on an authorisation failure. Video upload uses multipart form data; all other operations exchange JSON whose shape mirrors the backend Pydantic models in TypeScript types.

### *C. Security and Privacy*

Passwords are stored only as salted hashes; protected routes verify the JWT signature and expiry and return HTTP 401 otherwise; and each user can access only their own records. Batting videos show identifiable people, so videos are processed on project-controlled infrastructure and the language model runs locally, keeping user content away from third-party AI services. Consent is required before recording other people, including parental consent for minors.

## V. WORKFLOW

### *A. User Workflow*

Fig. 5 shows the application workflow. After registration or login, the backend issues a signed JWT that the client attaches to every protected request. The dashboard summarises the user's activity and weekly progress, and the user then chooses one of three paths: video analysis, the AI coach or skill learning. Each path returns to the dashboard, and analysis results are stored per user for later retrieval.

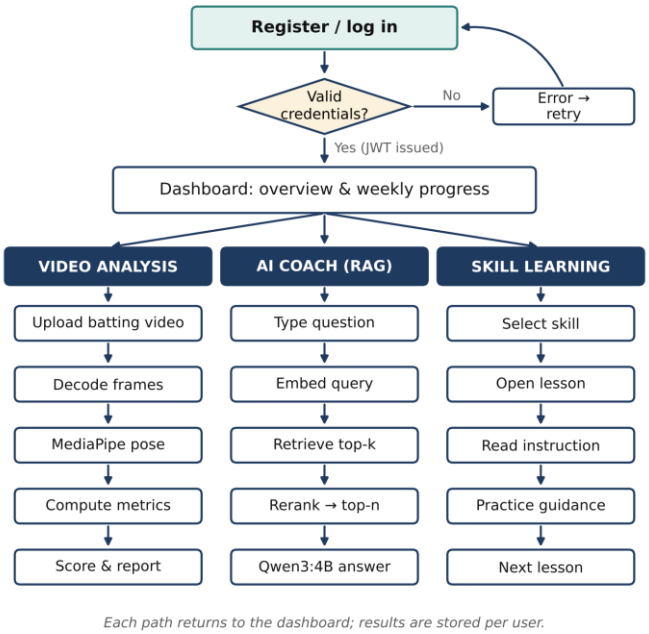



Fig. 5. Overall workflow: authentication, dashboard and the three feature paths.

### *B. Request Processing*

For video analysis, the client sends the file as multipart form data with the JWT; the API verifies the token and file type, stores the file, runs the analysis engine of Section III-A, saves the resulting record in MongoDB and returns the report as JSON, which the client renders as the analysis page. Previously analysed videos are retrieved through a separate authenticated request, and the dashboard aggregates the stored records into the overview and weekly progress.

For a coaching question, the client posts the question text; the API embeds it, queries ChromaDB for the top-$k$ chunks, reranks them, builds the prompt from the top-$n$ chunks and forwards it to the local Qwen3:4B model, then returns the generated answer. Lesson requests return the instructional content and practice guidance for the selected skill. Failures at any stage, such as an invalid token, an undecodable video or an unavailable model service, return an explicit error rather than a partial result.

## VI. RESULTS AND DISCUSSION

### *A. Functional Validation*

Table III summarises the observations recorded during development. All six core operations returned HTTP 200, and one analysis run processed every frame of a 1,654-frame clip whose container reported 30.03 fps, i.e. about 55.1 s of video. The 30.03 fps figure is the source frame rate of the input, not the system's processing speed. These observations establish that the components in Fig. 4 are integrated end to end; they do not by themselves establish the correctness of the returned content.

TABLE III
RECORDED FUNCTIONAL OBSERVATIONS

| Operation / item | Observation |
|---|---|
| Authentication (login) | HTTP 200 |
| Dashboard overview | HTTP 200 |
| Weekly progress | HTTP 200 |
| Video upload + analysis | HTTP 200 |
| Video retrieval | HTTP 200 |
| Coaching query (RAG) | HTTP 200 |
| Analysis run | 1,654 frames processed; source 30.03 fps (≈55.1 s) |

### B. Evaluation Protocol

Quantitative evaluation has not yet been carried out, so no accuracy, latency or user-study figures are reported. Table IV defines the protocol to be applied. Processing throughput is $FPS_p = N/T_p$ for $N$ frames processed in end-to-end time $T_p$; pose-measurement error is the mean absolute difference between predicted and manually annotated angles, reported per metric and per camera viewpoint; and retrieval quality is measured by Recall@k and mean reciprocal rank before and after reranking on a question set with manually identified relevant chunks.

TABLE IV
EVALUATION METRICS AND STATUS

| Metric | Procedure | Status |
|---|---|---|
| Processing time, $FPS_p$ | Server timestamps on clips of known length | Not measured |
| Pose angle MAE | Manual annotation of sampled frames; per viewpoint | Not measured |
| Recall@k, MRR | 30–50 questions with labelled relevant chunks | Not measured |
| Answer quality | Coach rates correctness, relevance, faithfulness (1–5) | Not measured |
| API latency | p50 / p95 over repeated calls per endpoint | Not measured |
| Usability | Task-based test + System Usability Scale | Not measured |
| Score validity | Agreement with qualified coaches' ratings | Not performed |

### C. Discussion

Pose-derived angles are most useful for comparing a player's own recordings taken from the same position, where viewpoint effects are held roughly constant. Aggregating them into one score embeds assumptions about ideal values that vary with stroke, phase, player and viewpoint, which is why the score is presented as an indicator rather than a judgement. For the assistant, retrieval and reranking should make answers more specific and traceable than ungrounded generation [5], [17], but answer quality is bounded by the coverage of the roughly one thousand indexed chunks and the capability of a 4-billion-parameter model. Finally, the analysis report and the assistant currently operate independently; passing structured analysis results to the RAG pipeline is the main step from a set of tools to an integrated coach.

### D. Limitations

- **Monocular viewpoint:** a single camera yields 2D projections; measured angles change with camera angle and perspective.
- **Depth and occlusion:** MediaPipe's $z$ is relative, not calibrated, and bat, pads and nets can hide joints.
- **Image quality:** low light, motion blur and compression reduce landmark reliability during fast swings.
- **Unvalidated score:** reference values and the score are not yet validated against expert assessment.
- **Incorrect advice:** the assistant can still produce wrong or incomplete guidance [7]; the interface should state that it does not replace a qualified coach.
- **Scope:** only batting is analysed in the version described.

## VII. CONCLUSION AND FUTURE WORK

CricketAI shows that open-source pose estimation, a local retrieve–rerank–generate pipeline and a modern web stack can be combined into a single cricket coaching assistant. Functional validation confirms that authentication, dashboard, video analysis and coaching queries execute end to end. Future work will (i) connect analysis results to the assistant so it can explain a user's weakest metrics, (ii) add bowling analysis, (iii) validate angle measurements against manual annotation and the score against coaches' ratings, (iv) segment strokes into phases and identify the stroke played, building on [9], so that reference values can depend on stroke and phase, (v) measure retrieval and answer quality with the protocol in Table IV, and (vi) conduct a usability study with club and college players. Any claim that the system improves player performance will require a controlled study.


## ACKNOWLEDGMENT

The authors thank the Department of Computer Science and Engineering, Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai, for its support, and the developers of MediaPipe, OpenCV, FastAPI, ChromaDB, the BGE models and the Qwen3 models.


## REFERENCES


[1] A. Badiola-Bengoa and A. Mendez-Zorrilla, "A systematic review of the application of camera-based human pose estimation in the field of sport and physical exercise," *Sensors*, vol. 21, no. 18, Art. no. 5996, 2021, doi: 10.3390/s21185996.

[2] C. Zheng, W. Wu, C. Chen, T. Yang, S. Zhu, J. Shen, N. Kehtarnavaz, and M. Shah, "Deep learning-based human pose estimation: A survey," *ACM Computing Surveys*, vol. 56, no. 1, Art. no. 11, 2023, doi: 10.1145/3603618.

[3] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. G. Yong, J. Lee, W.-T. Chang, W. Hua, M. Georg, and M. Grundmann, "MediaPipe: A framework for building perception pipelines," arXiv:1906.08172, 2019, doi: 10.48550/arXiv.1906.08172.

[4] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, "BlazePose: On-device real-time body pose tracking," arXiv:2006.10204, 2020, doi: 10.48550/arXiv.2006.10204.

[5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020. [Online]. Available: arXiv:2005.11401.

[6] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," arXiv:2312.10997, 2023, doi: 10.48550/arXiv.2312.10997.

[7] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, Art. no. 248, 2023, doi: 10.1145/3571730.

[8] A. Sen, K. Deb, P. K. Dhar, and T. Koshiba, "CricShotClassify: An approach to classifying batting shots from cricket videos using a convolutional neural network and gated recurrent unit," *Sensors*, vol. 21, no. 8, Art. no. 2846, 2021, doi: 10.3390/s21082846.

[9] H. U. R. Siddiqui, F. Younas, F. Rustam, E. S. Flores, J. B. Ballester, I. de la T. Diez, S. Dudley, and I. Ashraf, "Enhancing cricket performance analysis with human pose estimation and machine learning," *Sensors*, vol. 23, no. 15, Art. no. 6839, 2023, doi: 10.3390/s23156839.

[10] G. Thomas, R. Gade, T. B. Moeslund, P. Carr, and A. Hilton, "Computer vision for sports: Current applications and research topics," *Computer Vision and Image Understanding*, vol. 159, pp. 3–18, 2017, doi: 10.1016/j.cviu.2017.04.011.

[11] A. Toshev and C. Szegedy, "DeepPose: Human pose estimation via deep neural networks," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2014, pp. 1653–1660.

[12] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, and Y. Sheikh, "OpenPose: Realtime multi-person 2D pose estimation using part affinity fields," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 43, no. 1, pp. 172–186, 2021, doi: 10.1109/TPAMI.2019.2929257.

[13] Google AI Edge, "Pose landmark detection guide," MediaPipe Solutions documentation. [Online]. Available: https://ai.google.dev/edge/mediapipe/solutions/vision/pose_landmarker (accessed Oct. 2026).

[14] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proc. 2020 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781, doi: 10.18653/v1/2020.emnlp-main.550.

[15] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. 2019 Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992, doi: 10.18653/v1/D19-1410.

[16] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020, doi: 10.1109/TPAMI.2018.2889473.

[17] R. Nogueira and K. Cho, "Passage re-ranking with BERT," arXiv:1901.04085, 2019, doi: 10.48550/arXiv.1901.04085.

[18] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, D. Lian, and J.-Y. Nie, "C-Pack: Packed resources for general Chinese embeddings," in *Proc. 47th Int. ACM SIGIR Conf. Research and Development in Information Retrieval (SIGIR '24)*, Washington, DC, USA, 2024, doi: 10.1145/3626772.3657878.

[19] Qwen Team (A. Yang et al.), "Qwen3 technical report," arXiv:2505.09388, 2025, doi: 10.48550/arXiv.2505.09388.

[20] K. VanLehn, "The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems," *Educational Psychologist*, vol. 46, no. 4, pp. 197–221, 2011, doi: 10.1080/00461520.2011.611369.

[21] BAAI, "bge-base-en-v1.5 model card," Hugging Face. [Online]. Available: https://huggingface.co/BAAI/bge-base-en-v1.5 (accessed Oct. 2026).

[22] S. Ramírez, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com (accessed Oct. 2026).

[23] M. B. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015, doi: 10.17487/RFC7519.

[24] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, 2000.

[25] Chroma, "Chroma documentation." [Online]. Available: https://docs.trychroma.com (accessed Oct. 2026).

[26] Ollama, "Ollama documentation." [Online]. Available: https://docs.ollama.com (accessed Oct. 2026).

[27] Beanie, "Beanie ODM documentation." [Online]. Available: https://beanie-odm.dev (accessed Oct. 2026).